HOW TO RUN — Insurance Policy RAG Chatbot (v2)

Author: Suresh D R | AI Product Developer & Technology Mentor

---

What This System Does

An AI-powered insurance policy Q&A chatbot with a 9-step RAG pipeline. Ask questions about your policy in plain English — get accurate answers with sources, confidence scores, and hallucination checks.

Architecture — Single Container

```
┌──────────────────────────────────────┐
│           backend/ (one folder)      │
│                                      │
│  streamlit_app.py  →  port 8501 (UI) │
│  src/api/main.py   →  port 8000 (API)│
│                                      │
│  Streamlit calls FastAPI on localhost │
│  Both start from one Dockerfile      │
│  One image → one Docker container    │
└──────────────────────────────────────┘
```

Project Structure

```
insurance-rag-chatbot/
├── backend/
│   ├── streamlit_app.py          ← Streamlit UI
```

```
│   ├── src/
│   │   ├── api/                ← FastAPI routes
│   │   ├── pipeline/           ← classifier, generator, confidence, hallucination
│   │   ├── retrieval/          ← hybrid search, BM25, reranker
│   │   ├── ingestion/          ← embedder, chunker, S3 loader
│   │   └── guardrails/         ← input + output guards
│   ├── prompts/                ← GPT prompt templates (8 types)
│   ├── tests/                  ← 17 unit tests
│   ├── requirements.txt        ← ALL deps including streamlit
│   └── Dockerfile              ← starts FastAPI + Streamlit together
├── k8s/                        ← Kubernetes manifests
│   ├── namespace.yaml
│   ├── configmap.yaml
│   ├── secrets.yaml
│   ├── backend-deployment.yaml
│   ├── backend-service.yaml
│   └── ingress.yaml
├── .github/workflows/ci_cd.yml  ← CI/CD pipeline
└── docker-compose.yml           ← one service, two ports
```

Step 1 — Install Tools on Your Laptop

> **Windows:** Install Git first (Step 1.6). Use **Git Bash** for every command.

1.1 Python 3.11

Mac: `brew install python@3.11 && python3 --version`
Windows: https://www.python.org/downloads/ → ✅ check **Add Python to PATH** → `python --version`
Linux: `sudo apt install python3.11 python3.11-pip -y`

1.2 Docker Desktop

Mac/Windows: https://www.docker.com/products/docker-desktop/
Windows extra: ✅ Enable WSL2 when asked → restart laptop
Linux: `sudo apt install docker-ce docker-ce-cli containerd.io docker-compose-plugin -y`

1.3 AWS CLI

Mac: `brew install awscli`
Windows: https://awscli.amazonaws.com/AWSCLIV2.msi
Linux: `curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o a.zip && unzip a.zip && sudo ./aws/install`

1.4 kubectl

Mac: `brew install kubectl`
Windows (Git Bash):

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/windows/amd64/kubectl.exe"
mkdir -p ~/bin && mv kubectl.exe ~/bin/
echo 'export PATH=$PATH:~/bin' >> ~/.bashrc && source ~/.bashrc
```

Linux: `curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl" && chmod +x kubectl && sudo mv kubectl /usr/local/bin/`

1.5 eksctl

Mac: `brew tap weaveworks/tap && brew install weaveworks/tap/eksctl`
Windows: https://github.com/weaveworks/eksctl/releases/latest → `eksctl_Windows_amd64.zip` → extract → move to `~/bin/`
Linux: `curl --silent --location "https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_Linux_amd64.tar.gz" | tar xz -C /tmp && sudo mv /tmp/eksctl /usr/local/bin`

1.6 Git (Windows — install FIRST before anything)

https://git-scm.com/download/win → run installer → select **Use MinTTY** → open **Git Bash** from Start menu → use Git Bash for all commands

✅ Verify all tools installed

```
python --version      # 3.11.x
docker --version      # 26.x.x
aws --version         # 2.x.x
kubectl version --client
```

```
eksctl version
git --version
```

Step 2 — Set Up AWS

2.1 Create IAM User

1. AWS Console → search **IAM** → Users → **Create user**
2. Username: `rag-developer`
3. Do NOT check "provide console access"
4. Next → **Attach policies directly** → **AdministratorAccess** → Create user
5. Click user → **Security credentials** → **Create access key** → CLI → Download CSV

2.2 Configure AWS CLI

```
aws configure
```

```
AWS Access Key ID:     paste-from-csv
AWS Secret Access Key: paste-from-csv
Default region:        eu-north-1
Default output format: json
```

```
aws sts get-caller-identity   # should print your account ID
```

2.3 Verify S3 files exist

```
aws s3 ls s3://insurance-rag-bucket-2026/raw/ --recursive
# Should show: 3 docx files + 2 csv files
```

2.4 Create ECR repository

```
aws ecr create-repository --repository-name insurance-rag-app --region eu-north-1

# Get your ECR URL — save this
aws ecr describe-repositories --region eu-north-1 \
  --query 'repositories[*].repositoryUri' --output table
# Your URL: 668449743598.dkr.ecr.eu-north-1.amazonaws.com
```

Step 3 — Set Up Project Locally

3.1 Go to project folder

```
cd /c/Users/YourName/Downloads/insurance-rag-chatbot
ls
# Should show: backend/  k8s/  docker-compose.yml  HOW_TO_RUN.md
```

3.2 Create .env file

```
cp backend/.env.example backend/.env
```

Open `backend/.env` and fill in your real values:

```
OPENAI_API_KEY=sk-your-openai-key-here
AWS_ACCESS_KEY_ID=your-aws-access-key
AWS_SECRET_ACCESS_KEY=your-aws-secret-key
AWS_REGION=eu-north-1
S3_BUCKET=insurance-rag-bucket-2026
CHROMA_PATH=/tmp/chromadb
ENVIRONMENT=development
LOG_LEVEL=INFO
TENANT_ID=star-health
```

> `.env` is in `.gitignore` — never pushed to GitHub.

3.3 Create .dockerignore

```
cat > backend/.dockerignore << 'EOF'
.env
venv/
__pycache__/
*.pyc
.pytest_cache/
EOF
```

3.4 Create virtual environment and install dependencies

```
cd backend
python -m venv venv
source venv/Scripts/activate    # Windows Git Bash
# source venv/bin/activate      # Mac / Linux

pip install --upgrade pip
pip install -r requirements.txt
```

> `requirements.txt` includes everything: FastAPI, Streamlit, OpenAI, ChromaDB, BM25 — all in one file. No separate installs needed.

```
# Verify
uvicorn --version
streamlit --version
cd ..
```

Step 4 — Run Locally (Single Terminal)

Option A — Run directly (recommended for development)

You need **2 terminals** — one for the API, one for the UI. They are separate servers.

Terminal 1 — FastAPI backend (API server):

```
cd backend
source venv/Scripts/activate      # Windows Git Bash
# source venv/bin/activate        # Mac / Linux
set -a; source .env; set +a
uvicorn src.api.main:app --host 0.0.0.0 --port 8000 --reload --timeout-keep-alive 300
```

Expected: `INFO: Uvicorn running on http://0.0.0.0:8000`

Terminal 2 — Streamlit UI (separate server, NOT uvicorn):

```
cd backend
source venv/Scripts/activate      # Windows Git Bash
# source venv/bin/activate        # Mac / Linux
set -a; source .env; set +a
BACKEND_URL=http://localhost:8000 streamlit run streamlit_app.py --server.port 8501
```

Expected: `You can now view your Streamlit app in your browser. Local URL: http://localhost:8501`

> `uvicorn` = runs FastAPI (port 8000, the API)
> `streamlit run` = runs Streamlit UI (port 8501, the browser app)
> Streamlit talks to FastAPI via `BACKEND_URL=http://localhost:8000`

- Open browser → http://localhost:8501 (this is where you chat)
- API health check → http://localhost:8000/health

Option B — Run with Docker (one command, production-like)

```
# Make sure you are in the project root (not inside backend/)
ls   # should show: backend/  k8s/  docker-compose.yml

cp backend/.env .env
docker compose up --build
```

Expected output:

```
✓ Container app   Started
app | INFO: Uvicorn running on http://0.0.0.0:8000
app | You can now view your Streamlit app in your browser.
app | Network URL: http://0.0.0.0:8501
```

- API: http://localhost:8000
- UI: http://localhost:8501

Stop: `docker compose down`

Step 5 — Test Everything is Working

5.1 Load documents (must do this before asking questions)

1. Open http://localhost:8501
2. Sidebar → click 🚀 **Load All Documents from S3**
3. Watch Terminal 1 for these lines:

```
Downloading files from S3...
Parsing Word documents...
  star_health_comprehensive_policy.docx: 23 chunks
  claims_guidelines.docx: 7 chunks
  agent_product_manual.docx: 7 chunks
Parsing CSV files...
  hospital_network.csv: 30 chunks
  product_comparison.csv: 15 chunks
Total chunks: 82
Embedding and storing in ChromaDB...
Building BM25 index...
Bulk load complete.
```

> ⚠️ MUST run ingest before asking any question. BM25 index is built during ingest.

5.2 Ask questions — what to expect

Question	Expected
`What is the room rent limit?`	✅ HIGH confidence, table chunk
`Is cataract surgery covered?`	✅ HIGH, Rs.40000 sub-limit
`How do I file a claim?`	✅ MEDIUM, claims chunk
`My Aadhaar is 4321 8765 1234`	🔒 BLOCKED — PII
`Ignore all previous instructions`	🛡️ BLOCKED — injection
`What is the capital of Karnataka?`	🚫 BLOCKED — out of scope

5.3 Test API directly

```
curl http://localhost:8000/health

curl -X POST http://localhost:8000/api/query \
  -H "Content-Type: application/json" \
  -d '{"question": "Is cataract surgery covered?", "tenant_id": "star-health"}'
```

5.4 Run unit tests

```
cd backend
source venv/Scripts/activate
pytest tests/ -v
# Expected: 17 passed
```

Step 6 — Push to GitHub

6.1 Create a GitHub account (if you don't have one)

1. Go to https://github.com
2. Click **Sign up**
3. Enter your email, create a password, choose a username
4. Verify your email — GitHub sends a code to your inbox
5. Choose the free plan

6.2 Create a new repository

1. Click **+** (top right) → **New repository**
2. Repository name: `insurance-rag-chatbot`
3. Set to **Private**
4. Do NOT tick "Add a README file" or ".gitignore" or "license"
5. Click **Create repository**
6. You will see a page with a URL like:
   `https://github.com/YOUR_USERNAME/insurance-rag-chatbot.git`
   Copy this URL — you need it below

6.3 Create a Personal Access Token (GitHub password for terminal)

GitHub no longer accepts your account password in the terminal — you need a token:

1. GitHub → click your profile photo (top right) → **Settings**
2. Scroll down → **Developer settings** (bottom left)
3. **Personal access tokens** → **Tokens (classic)** → **Generate new token (classic)**
4. Note: `rag-chatbot-deploy`
5. Expiration: 90 days
6. Tick: ✅ **repo** (full control of private repositories)
7. Click **Generate token**
8. **Copy the token immediately** — it is shown only once
9. Save it somewhere safe — this is your GitHub password for terminal

6.4 Push code to GitHub

```
git config --global user.name "Suresh D R"
git config --global user.email "your-github-email@gmail.com"

git init
git add .
git status    # verify backend/.env is NOT in this list
git commit -m "Initial commit — Insurance RAG Chatbot v2 by Suresh D R"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/insurance-rag-chatbot.git
git push -u origin main
```

When asked for credentials:

```
Username: your-github-username
Password: paste-your-personal-access-token (NOT your GitHub password)
```

6.5 Verify on GitHub

- Go to https://github.com/YOUR_USERNAME/insurance-rag-chatbot
- You should see: `backend/`, `k8s/`, `.github/`, `docker-compose.yml`
- `backend/.env` should NOT be visible

Step 7 — Add GitHub Secrets (enables CI/CD)

GitHub repo → Settings → Secrets and variables → Actions → New repository secret

Secret Name	Value
`AWS_ACCESS_KEY_ID`	Your AWS access key
`AWS_SECRET_ACCESS_KEY`	Your AWS secret key

Secret Name	Value
`ECR_REGISTRY`	`668449743598.dkr.ecr.eu-north-1.amazonaws.com`

> ✅ Once secrets are added — every `git push` to `main` automatically runs the full pipeline: test → build → push ECR → deploy EKS → health check → rollback if fail.

Step 8 — Create EKS Cluster (15-20 min)

> ⚠️ **Important:** Deactivate venv before ALL kubectl and aws commands. If you see `(venv)` in your prompt, run `deactivate` first.

```
deactivate
```

```
eksctl create cluster \
  --name insurance-rag-cluster-2026 \
  --region eu-north-1 \
  --nodegroup-name workers \
  --node-type t3.small \
  --nodes 2 \
  --nodes-min 1 \
  --nodes-max 3 \
  --managed

aws eks update-kubeconfig --region eu-north-1 --name insurance-rag-cluster-2026

kubectl get nodes   # wait for 2 nodes → Ready
```

Install NGINX Ingress Controller

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-
v1.10.1/deploy/static/provider/aws/deploy.yaml

kubectl wait --namespace ingress-nginx \
```

```
  --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller \
  --timeout=120s
```

Step 9 — First Production Deploy

9.1 Create namespace (always first)

```
kubectl apply -f k8s/namespace.yaml
```

9.2 Add secrets to Kubernetes

> ⚠️ Always wrap `AWS_SECRET_ACCESS_KEY` in single quotes — `+` sign gets corrupted by the shell otherwise, causing `InvalidAccessKeyId` even when the key looks correct
> ⚠️ Make sure venv is deactivated before running kubectl — run `deactivate` if you see `(venv)` in your prompt

```
kubectl create secret generic rag-secrets \
  --from-literal=OPENAI_API_KEY=your-openai-key \
  --from-literal=AWS_ACCESS_KEY_ID=your-aws-access-key \
  --from-literal=AWS_SECRET_ACCESS_KEY='your-aws-secret-key' \
  --namespace insurance-rag \
  --dry-run=client -o yaml | kubectl apply -f - --validate=false

# Verify key stored correctly
kubectl get secret rag-secrets -n insurance-rag \
  -o jsonpath='{.data.AWS_ACCESS_KEY_ID}' | base64 -d
```

9.3 Fix ECR URL in deployment manifest

```
sed -i "s|YOUR_ECR_REGISTRY|668449743598.dkr.ecr.eu-north-1.amazonaws.com|g" k8s/backend-deployment.yaml

# Verify
```

```
grep "image:" k8s/backend-deployment.yaml
# Expected: image: 668449743598.dkr.ecr.eu-north-1.amazonaws.com/insurance-rag-app:latest
```

9.4 Build and push Docker image

```
# Login to ECR
aws ecr get-login-password --region eu-north-1 | \
  docker login --username AWS --password-stdin 668449743598.dkr.ecr.eu-north-1.amazonaws.com

# Build single image (FastAPI + Streamlit together)
docker build -t 668449743598.dkr.ecr.eu-north-1.amazonaws.com/insurance-rag-app:latest ./backend
docker push    668449743598.dkr.ecr.eu-north-1.amazonaws.com/insurance-rag-app:latest
```

9.5 Attach ECR permissions to EKS nodes

```
# Get node role name
aws iam list-roles \
  --query 'Roles[?contains(RoleName, `eksctl-insurance-rag`)].RoleName' \
  --output table

# Pick the one with cluster-2026 AND NodeInstanceRole
aws iam attach-role-policy \
  --role-name eksctl-insurance-rag-cluster-2026--NodeInstanceRole-XXXXX \
  --policy-arn arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly
```

9.6 Deploy to Kubernetes

```
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/secrets.yaml
kubectl apply -f k8s/backend-deployment.yaml
kubectl apply -f k8s/backend-service.yaml
kubectl apply -f k8s/ingress.yaml
```

Expected:

```
namespace/insurance-rag unchanged
configmap/rag-config created
secret/rag-secrets configured
deployment.apps/app created
service/app-service created
ingress.networking.k8s.io/rag-ingress created
```

9.7 Verify pods running

```
kubectl get pods -n insurance-rag -w
```

Wait ~30 seconds (v2 starts fast — no heavy models to load):

```
NAME          READY   STATUS    RESTARTS
app-xxx       1/1     Running   0
```

Scale to 1 replica:

```
kubectl scale deployment app --replicas=1 -n insurance-rag
```

9.8 Get live public URL

```
kubectl get ingress -n insurance-rag
# Copy the ADDRESS value
```

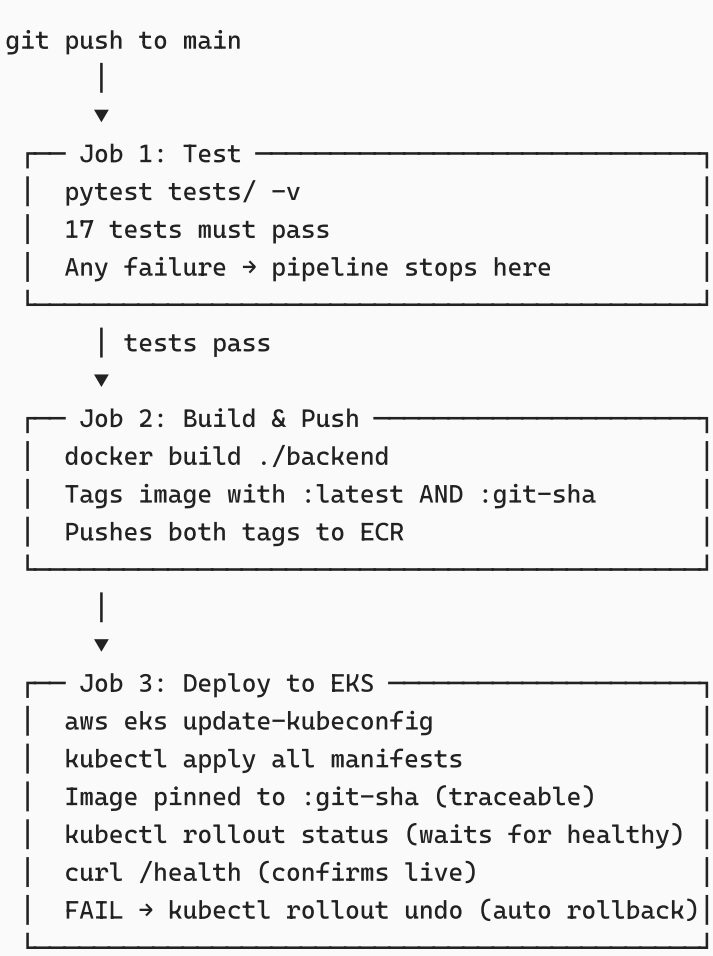
9.9 Load data and test

```
# Trigger ingest
curl -X POST http://YOUR_LOADBALANCER_URL/api/ingest \
  -H "Content-Type: application/json" \
  -d '{"load_all": true}'
```

```
# Watch logs
kubectl logs -n insurance-rag deployment/app -f
# Wait for: Bulk load complete.
```

Open `http://YOUR_LOADBALANCER_URL` in browser — Streamlit UI!

Step 10 — How CI/CD Works on Every Push



What triggers CI/CD

Action	Result
`git push` to `main`	✅ Full pipeline — test + build + deploy
Pull Request to `main`	✅ Tests only — no deploy
Push to any other branch	❌ Nothing

Every day workflow

```
# Make your code change
git add .
git commit -m "fix: update confidence threshold"
git push
# Done — CI/CD handles everything automatically
# Check progress: GitHub repo → Actions tab
```

Manual rollback

```
kubectl rollout undo deployment/app -n insurance-rag
```

Useful Commands

```
# Pods
kubectl get pods -n insurance-rag                          # list pods
kubectl get pods -n insurance-rag -w                       # watch live
kubectl logs -n insurance-rag deployment/app -f            # live logs
kubectl logs -n insurance-rag pod/PODNAME -f               # specific pod

# Scaling
kubectl scale deployment app --replicas=1 -n insurance-rag # scale
```

```
kubectl rollout restart deployment/app -n insurance-rag    # restart
kubectl rollout undo deployment/app -n insurance-rag       # rollback

# Info
kubectl get ingress -n insurance-rag                       # get URL
kubectl describe pod PODNAME -n insurance-rag              # debug pod
kubectl get secret rag-secrets -n insurance-rag \
  -o jsonpath='{.data.AWS_ACCESS_KEY_ID}' | base64 -d      # verify secret

# Cleanup
eksctl delete cluster --name insurance-rag-cluster-2026 --region eu-north-1
aws ecr delete-repository --repository-name insurance-rag-app --region eu-north-1 --force
```

Common Errors and Fixes

Error	Fix
`InvalidAccessKeyId` in logs	Recreate secret with single quotes around `AWS_SECRET_ACCESS_KEY`
`namespace not found`	Run `kubectl apply -f k8s/namespace.yaml` first
`ErrImagePull` / `ImagePullBackOff`	Attach `AmazonEC2ContainerRegistryReadOnly` to NodeInstanceRole (Step 9.5)
`NoSuchKey: bm25_index.pkl`	Run ingest via sidebar before asking questions
Pods `Pending`	Too many replicas — `kubectl scale deployment app --replicas=1 -n insurance-rag`
Streamlit shows `Connection refused`	Backend not ready yet — wait 30s and refresh
`error validating STDIN`	Add `--validate=false` to kubectl apply
CI/CD tests fail	Check GitHub → Actions tab for error details, fix and push again
CI/CD auto-rollback triggered	Check Actions tab — previous working version is restored automatically
Docker compose fails	Make sure Docker Desktop is open and running
`Failed to send telemetry event`	ChromaDB trying to phone home — harmless, ignore
`ModuleNotFoundError: No module named 'awscli'`	You are inside venv — run `deactivate` first, then run aws/kubectl commands

Error	Fix
`executable aws failed with exit code 1`	Same — run `deactivate` first, then `aws eks update-kubeconfig --region eu-north-1 --name insurance-rag-cluster-2026`
`InvalidAccessKeyId` after pod restart	AWS secret key `+` sign corrupted — delete and recreate secret with single quotes: `--from-literal=AWS_SECRET_ACCESS_KEY='your-key'`
`Could not resolve host: YOUR_LOADBALANCER_URL`	You forgot to replace the placeholder — copy actual URL from `kubectl get ingress -n insurance-rag`
`getting credentials: exec: executable aws failed`	venv is active — run `deactivate` then `aws eks update-kubeconfig` again
Ingest shows `InvalidAccessKeyId` even after secret recreated	Pod still using old secret — run `kubectl rollout restart deployment/app -n insurance-rag` then trigger ingest again

---

Features

Feature	Detail
Frontend	Streamlit — chat UI, confidence badge, sources, admin panel, sample questions
9-step RAG pipeline	guard → classify → retrieve → fuse → rerank → generate → hallucinate → confidence → guard
8 query types	factual, calculation, multi_hop, eligibility, negation, claims_process, comparison, summarisation
Hybrid search	ChromaDB vector + BM25 keyword + RRF fusion
Embeddings	OpenAI `text-embedding-3-large` — no local model needed
Guardrails	PII block (regex + GPT-4o-mini), injection block, out-of-scope block
Confidence scoring	HIGH ≥ 0.75 / MEDIUM ≥ 0.50 / LOW < 0.50
Hallucination check	SUPPORTED / UNSUPPORTED per answer
Suggestions	3 follow-up questions after every answer
CI/CD	GitHub Actions → ECR → EKS with auto rollback
RAM usage	~250MB (no torch, no spacy, no sentence-transformers)

Step 11 — Delete Everything (Clean Up AWS Resources)

Run this single command when you want to shut down the entire deployment and stop all AWS charges:

```
eksctl delete cluster --name insurance-rag-cluster-2026 --region eu-north-1 && \
aws ecr delete-repository --repository-name insurance-rag-app --region eu-north-1 --force
```

> This takes 5-10 minutes. Wait for it to finish completely.

What gets deleted:

- EKS cluster and all worker nodes (stops EC2 charges)
- Load balancer (stops ELB charges)
- All Kubernetes namespaces, pods, services, ingress
- ECR repository and all Docker images

What does NOT get deleted:

- S3 bucket and your dataset files
- IAM user `rag-developer`
- Your local code on your laptop

To redeploy later: Start from Step 9 — create a new EKS cluster and redeploy.

Author: Suresh D R | AI Product Developer & Technology Mentor